

Использование Pandas для описания наборов данных

Цель видео

Научиться использовать Pandas для получения общей статистики по наборам данных.

Что такое Pandas?

Pandas — это одна из **мощных библиотек Python** для **анализа данных**.
Используется для структурирования, изменения и манипулирования данными.
Построена на пакете NumPy, и его ключевая структура данных — **DataFrame**.
DataFrame позволяет нам хранить табличные данные в **строках** и **столбцах**.

Основные функции Pandas:

- Загрузка данных:

```
import pandas as pd  
df = pd.read_csv('data.csv')
```

- Получение общей статистики:

```
print(df.describe())
```

- Информация о структуре данных:

```
print(df.info())
```

Примеры использования Pandas

`read_csv`, `read_excel`

Наиболее распространённым способом создания DataFrame является чтение данных из различных форматов файлов, таких как CSV, Excel и текстовые файлы.

```
df_1 = pd.read_csv('filename.txt')
df_2 = pd.read_csv('filename.csv')
df_3 = pd.read_excel('filename.xlsx')
```


Примеры использования Pandas

Функция `read_csv()` считывает набор данных и просматривает 5 верхних строк.

```
df=pd.read_csv("/content/train.csv")

[ ] df.head(5)
```

	battery_power	blue	clock_speed	dual_sim	fc	four_g	int_memory	m_dep	mobile_wt	n_cores	...	px_height
0	842	0	2.2	0	1	0	7	0.6	188	2	...	20
1	1021	1	0.5	1	0	1	53	0.7	136	3	...	905
2	563	1	0.5	1	2	1	41	0.9	145	5	...	1263
3	615	1	2.5	0	0	0	10	0.8	131	6	...	1216
4	1821	1	1.2	0	13	1	44	0.6	141	2	...	1208

5 rows × 21 columns

info()

Функция `info()` помогает собирать информацию о данных:

- количество строк и столбцов
- название столбцов
- тип данных столбцов
- количество ненулевых точек данных в каждом из столбцов

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 2000 entries, 0 to 1999  
Data columns (total 21 columns):  
 #   Column                Non-Null Count  Dtype    
---  -  
 0   battery_power         2000 non-null   int64    
 1   blue                  2000 non-null   int64    
 2   clock_speed           2000 non-null   float64   
 3   dual_sim              2000 non-null   int64    
 4   fc                    2000 non-null   int64    
 5   four_g                2000 non-null   int64    
 6   int_memory            2000 non-null   int64    
 7   m_dep                 2000 non-null   float64   
 8   mobile_wt             2000 non-null   int64    
 9   n_cores               2000 non-null   int64
```

columns

Функция `columns` печатает имена всех столбцов нашего DataFrame:

```
df.columns
```

```
df.columns
```

```
Index(['battery_power', 'blue', 'clock_speed', 'dual_sim', 'fc', 'four_g',  
      'int_memory', 'm_dep', 'mobile_wt', 'n_cores', 'pc', 'px_height',  
      'px_width', 'ram', 'sc_h', 'sc_w', 'talk_time', 'three_g',  
      'touch_screen', 'wifi', 'price_range'],  
      dtype='object')
```


iloc()

Функция `iloc()` используется для фильтрации/извлечения данных с использованием индексов строк и столбцов в качестве параметров.

```
df.iloc[row_index, column_index]
```

Извлечение первой строки DataFrame.

```
df.iloc[0,0]
```

842

```
df.iloc[0]
```

battery_power	842.0
blue	0.0
clock_speed	2.2
dual_sim	0.0
fc	1.0
four_g	0.0
int_memory	7.0
m_dep	0.6
mobile_wt	188.0

loc()

Функция loc() почти аналогична функции iloc(). Используя функцию, вы можете указать индекс строки и имя столбца, который вы хотите извлечь. Ниже приведён синтаксис:

```
df.loc[row_index, column_index]
```

```
df.loc[0:2, ['blue', 'dual_sim']]
```

	blue	dual_sim
0	0	0
1	1	1
2	1	1



describe()

Функцию `description()` можно использовать для предоставления некоторых основных статистических показателей всего набора данных или нескольких столбцов в зависимости от потребностей пользователя.

df.describe()

	battery_power	blue	clock_speed	dual_sim	fc	four_g	int_memory	m_dep	mobile_wt	n_cores	...	px_height
count	2000.000000	2000.0000	2000.000000	2000.000000	2000.000000	2000.000000	2000.000000	2000.000000	2000.000000	2000.000000	...	2000.000000
mean	1238.518500	0.4950	1.522250	0.509500	4.309500	0.521500	32.046500	0.501750	140.249000	4.520500	...	645.108000
std	439.418206	0.5001	0.816004	0.500035	4.341444	0.499662	18.145715	0.288416	35.399655	2.287837	...	443.780811
min	501.000000	0.0000	0.500000	0.000000	0.000000	0.000000	2.000000	0.100000	80.000000	1.000000	...	0.000000
25%	851.750000	0.0000	0.700000	0.000000	1.000000	0.000000	16.000000	0.200000	109.000000	3.000000	...	282.750000
50%	1226.000000	0.0000	1.500000	1.000000	3.000000	1.000000	32.000000	0.500000	141.000000	4.000000	...	564.000000
75%	1615.250000	1.0000	2.200000	1.000000	7.000000	1.000000	48.000000	0.800000	170.000000	7.000000	...	947.250000
max	1998.000000	1.0000	3.000000	1.000000	19.000000	1.000000	64.000000	1.000000	200.000000	8.000000	...	1960.000000

8 rows × 21 columns

drop()

drop() отбрасывает/удаляет некоторые ненужные столбцы/строки.

Аргумент оси используется, чтобы указать, хотим ли мы удалить строки или столбцы DataFrame:
Axis = 0 для строк и Axis = 1 для столбцов.

Видно, что общее количество столбцов уменьшилось с 21 до 20.

```
df.drop('blue', axis = 1)
```

	battery_power	clock_speed	dual_sim	fc	four_g	int_memory	m_dep	mobile_wt
0	842	2.2	0	1	0	7	0.6	188
1	1021	0.5	1	0	1	53	0.7	136
2	563	0.5	1	2	1	41	0.9	145
3	615	2.5	0	0	0	10	0.8	131
4	1821	1.2	0	13	1	44	0.6	141
...	...	...	...	...	...	...	...	...
1995	794	0.5	1	0	1	2	0.8	106
1996	1965	2.6	1	0	0	39	0.2	187
1997	1911	0.9	1	1	1	36	0.7	108
1998	1512	0.9	0	4	1	46	0.1	145
1999	510	2.0	1	5	1	45	0.9	168

2000 rows × 20 columns

shape

Функция shape выводит количество строк и столбцов DataFrame.

```
df.shape
```

```
df.shape
```

```
(2000, 21)
```


isnull()

Функция `isnull()` используется для проверки наличия нулевых значений в `DataFrame`.

Возвращает `True` (или `1`) для нулевых значений и `False` (или `0`) для ненулевых значений.

```
df.isnull().sum()
```

```
df.isnull().sum()
```

battery_power	0
blue	0
clock_speed	0
dual_sim	0
fc	0
four_g	0
int_memory	0
m_dep	0
mobile_wt	0
n_cores	0
pc	0
ox_height	0

groupby()

Функция groupby() суммирует данные.

```
df.groupby(['column1']).aggregate_function()
```

```
data4 = {  
    'co2': [95, 90, 99, 104, 105, 94, 99, 104],  
    'model': ['Citigo', 'Fabia', 'Fiesta', 'Rapid', 'Focus', 'Mondeo', 'Octavia', 'B-Max'],  
    'car': ['Skoda', 'Skoda', 'Ford', 'Skoda', 'Ford', 'Ford', 'Skoda', 'Ford']  
}
```

```
df4 = pd.DataFrame(data4)
```

```
df4
```

	co2	model	car
0	95	Citigo	Skoda
1	90	Fabia	Skoda
2	99	Fiesta	Ford
3	104	Rapid	Skoda
4	105	Focus	Ford
5	94	Mondeo	Ford
6	99	Octavia	Skoda
7	104	B-Max	Ford

groupby()

Функция **groupby()** группирует набор данных по столбцу «автомобиль» и находит средние выбросы CO₂ для автомобилей.

```
print(df.groupby(["car"]).mean())
```

```
print(df4.groupby(["car"]).mean())
```

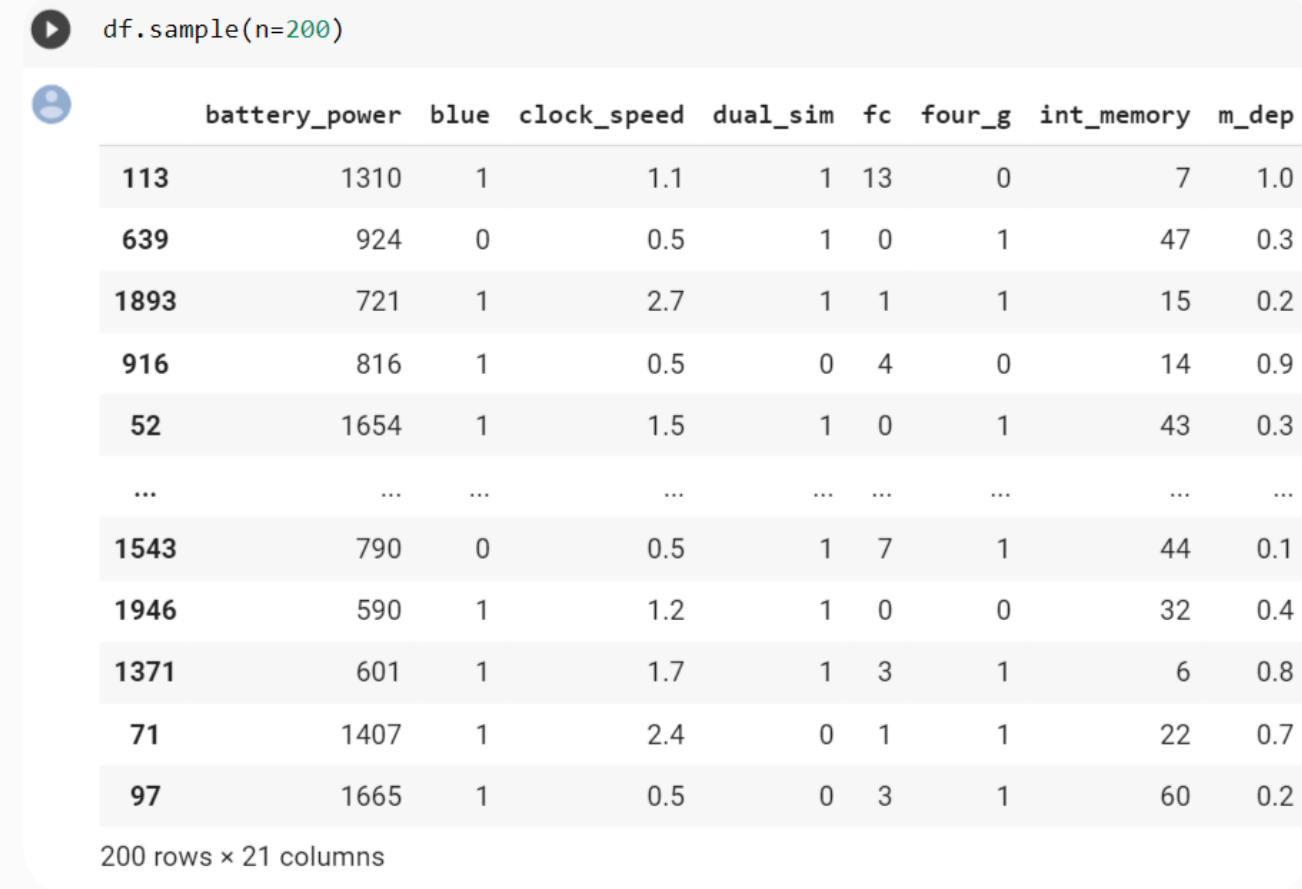
	co2
car	
Ford	100.5
Skoda	97.0

sample()

Функция `sample()` выбирает случайным образом строки из `DataFrame`.

Следующий код показывает, как случайным образом выбрать 200 строк из `DataFrame`:

```
df.sample(n=200)
```



The image shows a Jupyter Notebook interface. At the top, there is a code cell with the command `df.sample(n=200)`. Below it, a user icon is visible. The main part of the image is a table representing a sample of 200 rows from a DataFrame. The table has 9 columns: an index column, `battery_power`, `blue`, `clock_speed`, `dual_sim`, `fc`, `four_g`, `int_memory`, and `m_dep`. The rows are numbered 113, 639, 1893, 916, 52, followed by an ellipsis, then 1543, 1946, 1371, 71, and 97. The last row of the table is labeled "200 rows x 21 columns".

	battery_power	blue	clock_speed	dual_sim	fc	four_g	int_memory	m_dep
113	1310	1	1.1	1	13	0	7	1.0
639	924	0	0.5	1	0	1	47	0.3
1893	721	1	2.7	1	1	1	15	0.2
916	816	1	0.5	0	4	0	14	0.9
52	1654	1	1.5	1	0	1	43	0.3
...	...	...	...	...	...	...	...	...
1543	790	0	0.5	1	7	1	44	0.1
1946	590	1	1.2	1	0	0	32	0.4
1371	601	1	1.7	1	3	1	6	0.8
71	1407	1	2.4	0	1	1	22	0.7
97	1665	1	0.5	0	3	1	60	0.2

200 rows x 21 columns

unique()

Функция unique() ищет уникальные значения в столбце.

```
df['column'].unique()
```

```
df['battery_power'].unique()
```

```
array([ 842, 1021,  563, ..., 1139, 1467,  858])
```

```
len(df['battery_power'].unique())
```

```
1094
```

replace()

Функция `replace()` заменяет значения столбца значениями, определёнными пользователем. Нам нужно предоставить старые значения и новые значения в качестве аргументов функции `replace()`.

```
df.replace.(old value, new value)
```

```
df2 = {  
    "Array_1": [49.50, 70],  
    "Array_2": [65.1, 49.50]  
}
```

```
data = pd.DataFrame(df2)  
data.head()
```

	Array_1	Array_2
0	49.5	65.1
1	70.0	49.5

```
print(data.replace(49.50, 60))
```

	Array_1	Array_2
0	60.0	65.1
1	70.0	60.0

rank()

Функция **rank()** присваивает ранг столбцам DataFrame.

df.rank()

Ранг, связанный со столбцом, увеличивается по мере увеличения значения в столбце. По возрастанию, по умолчанию.

```
[ ] df4['rank'] = df4['co2'].rank()
```

df4

	co2	model	car	rank
0	95	Citigo	Skoda	3.0
1	90	Fabia	Skoda	1.0
2	99	Fiesta	Ford	4.5
3	104	Rapid	Skoda	6.5
4	105	Focus	Ford	8.0
5	94	Mondeo	Ford	2.0
6	99	Octavia	Skoda	4.5
7	104	B-Max	Ford	6.5